# Solving Equations in One Variable

Chapter 2 of Numerical Analysis

Yiping Cheng
ypcheng@bjtu.edu.cn
22-Aug-2022

School of Electronic and Information Engineering
Beijing Jiaotong University

# Introduction

Let $f$ be a real-valued function of a real variable. A number $r$, for which $f(r) = 0$ is called a **root** or **solution** of that equation or a **zero** of $f$. For example, the function $f(x) = 6x^2 - 7x + 2$ has $\frac{1}{2}$ and $\frac{2}{3}$ as zeros. For another example, the function $g(x) = \cos(3x) - \cos(7x)$ has not only the obvious zero 0, but every integer multiple of $\pi/5$ and of $\pi/2$ as well.

Why is locating roots important? Frequently, the solution to a scientific problem is a number about which we have little information other than that it satisfies some equation. Since every equation can be written so that a function stands on one side and zero on the other, the desired number must be a zero of the function. Thus, if we possess an arsenal of methods for locating zeros of functions, we shall be able to solve such problems.

Throughout this chapter, we assume that $f$ is continuous.

# Chapter Contents


1 Bracketing Methods
- Bisection Method
- Regula Falsi Method
- Ridders' Method

2 Open Methods
- Newton's Method
- Secant Method

3 Hybrid Methods

# Bracketing Methods

# Introduction to bracketing methods

The intermediate value theorem guarantees that for a continuous function whose domain contains the interval $[a, b]$ takes any value between $f(a)$ and $f(b)$ at some point in the interval. Bracketing methods rely on the fact that if the signs of $f(a)$ and $f(b)$ are different, then there exists a root in between $[a, b]$.

Due to this fact, the bracketing methods maintain that, during the iteration process, the variables $a$ and $b$ must be such that $f(a)$ and $f(b)$ have different signs, ensuring that $[a, b]$ contains a root.

This also means that the user needs to supply a bracketing interval before using these methods, which is a disadvantage.

# Bisection method

**Algorithm 2.1:** Bisection

**Input:** $f, a, b, T_{ol}, M_{axiter}$ with $f(a) \cdot f(b) < 0$
**Output:** $a, b$ with $[a, b]$ containing zero of $f$

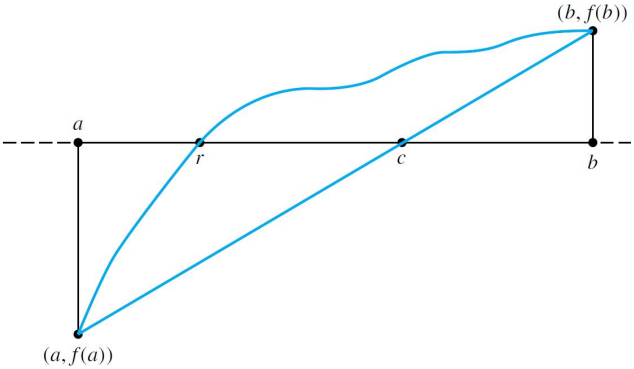
```
k = 0
while |a − b| > T_OL and k < M_axiter do
    c = (a + b)/2
    if f(c)==0 then
        a = b = c
        return
    else if f(c) · f(a) > 0 then
        a = c
    else
        b = c
    k = k + 1
```

Features of Bisection:

- Denote the values of $a$, $b$ at the $k$th iteration by $a_k$, $b_k$. Then $b_{k+1} - a_{k+1} = \frac{b_k - a_k}{2}$, i.e. the length of the bracketing interval halves after every iteration. Thus the algorithm is guaranteed to converge.
- It utilizes only the signs of $f(a_k)$ and $f(b_k)$, but not their values. This suggests room for further improvement.

# Regula Falsi method

Rather than selecting the midpoint of each interval, this method uses the point where the secant line intersects the x-axis.




The secant line over the interval $[a, b]$ is the chord between $(a, f(a))$ and $(b, f(b))$. Let it intersect the x-axis at $(c, 0)$. Then $\frac{f(b)}{b-c} = \frac{f(b)-f(a)}{b-a}$, and we have

# Regula Falsi method

**Algorithm 2.2:** Regula Falsi

**Input:** $f, a, b, T_{ol}, M_{axiter}$ with $f(a) \cdot f(b) < 0$
**Output:** $a, b$ with $[a, b]$ containing zero of $f$

```
k = 0
while |a − b| > T_OL and k < M_axiter do
    c = (a f(b) − b f(a)) / (f(b) − f(a))
    if f(c)==0 then
        a = b = c
        return
    else if f(c) · f(a) > 0 then
        a = c
    else
        b = c
    k = k + 1
```

Features of Regula Falsi:

- Utilizes the values of $f(a_k)$ and $f(b_k)$, therefore usually converge faster than bisection in practice.
- The length of the bracketing interval no longer necessarily halves after each iteration. Thus the algorithm is not guaranteed to converge.
- Can we combine the safety of bisection and the performance of Regula Falsi?

# Ridders' method

Ridders' method is one of the most elegant algorithms in numerical analysis. It is based on the following ideas:

- It assumes that $f(x)$ can be approximated by $\kappa e^{\lambda x}(x-c)$ in the interval $[a,b]$. And then it computes the values of $\kappa, \lambda, c$ by using the values of $f(a)$, $f(b)$, $f(m)$ where $m = \frac{a+b}{2}$.
- If $f(m)$ and $f(c)$ have different signs, then let $[m,c]$ (if $m < c$) or $[c,m]$ (if $c < m$) be the next bracketing interval. Otherwise, let $[a,c]$ or $[c,b]$ be the next interval. The interval's length at least halves, so the algorithm is guaranteed to converge.

# Derivation of Ridders' method

Let $d = \frac{b-a}{2}$. Then

$$\left.\begin{array}{rcl} f(a) & = & \kappa e^{\lambda a}(a-c) \\ f(m) & = & \kappa e^{\lambda m}(m-c) \\ f(b) & = & \kappa e^{\lambda b}(b-c) \end{array}\right\} \Rightarrow \left\{\begin{array}{rcl} e^{2\lambda d} f(a) & = & \kappa e^{\lambda b}(a-c) \\ e^{\lambda d} f(m) & = & \kappa e^{\lambda b}(m-c) \\ f(b) & = & \kappa e^{\lambda b}(b-c) \end{array}\right.$$

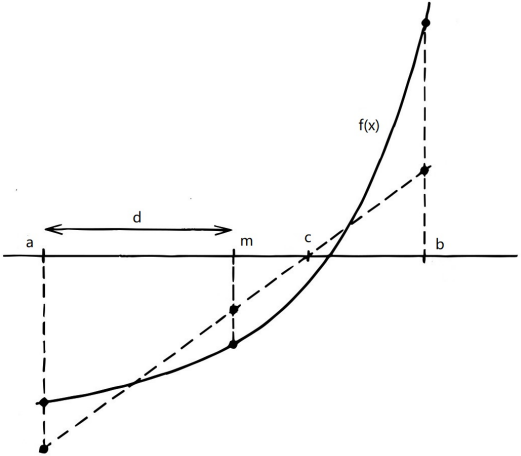
$$e^{2\lambda d} f(a) - 2e^{\lambda d} f(m) + f(b) = 0$$

$$e^{\lambda d} = \frac{f(m) \pm \sqrt{W}}{f(a)} \text{ where } W = f^2(m) - f(a)f(b).$$

Since $f(a)$ and $f(b)$ have different signs, $W > 0$, and

$$e^{\lambda d} = \frac{f(m) + \text{sign}[f(a)]\sqrt{W}}{f(a)}.$$

Thus $e^{-\lambda d} = \frac{f(m) - \textbf{sign}[f(a)]\sqrt{W}}{f(b)}$.



# Derivation of Ridders' method (continued)

$$e^{-\lambda d} f(b) - f(m) = \kappa e^{\lambda m} d$$

$$\kappa e^{\lambda m} = \frac{e^{-\lambda d} f(b) - f(m)}{d}$$

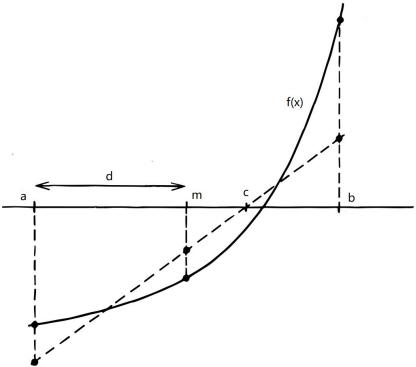
$$m - c = \frac{f(m)}{\kappa e^{\lambda m}} = \frac{f(m)\, d}{e^{-\lambda d} f(b) - f(m)}$$

$$c = m - \frac{f(m)\, d}{e^{-\lambda d} f(b) - f(m)} = m + d \cdot \frac{f(m)}{f(m) - e^{-\lambda d} f(b)}$$

$$= m + d \cdot \frac{f(m)}{\operatorname{sign}[f(a)]\sqrt{W}}$$

$$= m + d \cdot \operatorname{sign}[f(a)] \cdot \frac{f(m)}{\sqrt{W}}.$$

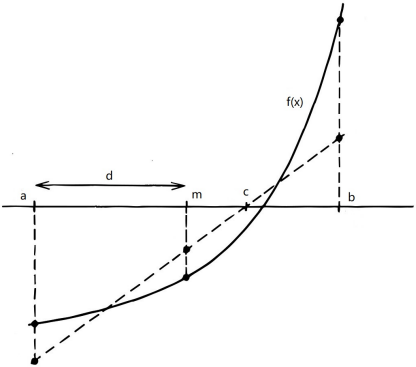
Since $|f(m)| < \sqrt{W}$, $c$ must lie in the interval $(a, b)$.



# Derivation of Ridders' method (continued)

$$c = m + d \cdot \mathbf{sign}[f(a)] \cdot \frac{f(m)}{\sqrt{f^2(m) - f(a)f(b)}}.$$

- If $f(m)$ and $f(c)$ have different signs, then let $[m, c]$ (if $m < c$) or $[c, m]$ (if $c < m$) be the next bracketing interval. Obviously $|m - c| < \frac{1}{2}|b - a|$.
- If $f(m)$ and $f(c)$ have same signs, then we have two cases:
  - $f(m)$, $f(c)$, $f(a)$ have same signs. Then $f(c)$, $f(b)$ have different signs and $c > m$, the interval $[c, b]$ is a bracketing interval and obviously $|b - c| < \frac{1}{2}|b - a|$.
  - $f(m)$, $f(c)$, $f(b)$ have same signs. Then $f(a)$, $f(c)$ have different signs and $c < m$, the interval $[a, c]$ is a bracketing interval and obviously $|c - a| < \frac{1}{2}|b - a|$.



# Open Methods

# Introduction to open methods

Unlike bracketing methods, open methods require only a single starting value (or two starting values), that do not need to bracket the root. As such, open methods can diverge; and they cannot be used alone. They must be combined with other techniques to ensure convergence. However, when they converge, they converge faster than bracketing methods.

There is another angle to view open methods. Let $s(x) = f^2(x)$. Then any **root of** $f(x) = 0$ is a **minimizer of** $s(x)$. And we can use minimization (optimization) algorithms to solve root-finding problems. In optimization, there is no concept of bracketing, where it is the most natural to supply initial point(s).

# Newton's method

In Newton's method, the function $f$ is assumed to be differentiable.

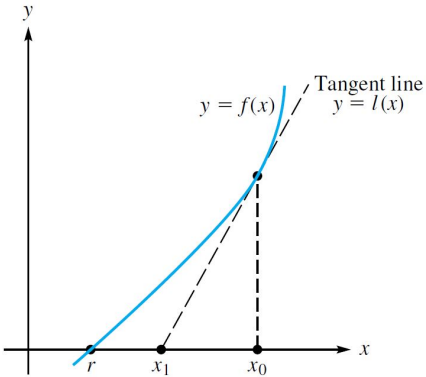
Let $x_0$ be the initial value, then around $x_0$, the function

$$l(x) = f(x_0) + f'(x_0)(x - x_0)$$

is a first-order approximation of $f(x)$. We take the zero of $l$ as an approximation to the zero of $f$. The zero of $l$ is easily found:

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}.$$

This is called a **Newton iteration**, and this iteration is repeated again and again and we arrive at the following algorithm.



# Newton's method

**Algorithm 2.3:** Newton

**Input:** $f, f', x, M_{axiter}$

**Output:** $x$ which is an approximate zero of $f$

$k = 0$

**while** $k < M_{axiter}$ **do**

  $x = x - \frac{f(x)}{f'(x)}$

  $k = k + 1$

There is no guarantee that Newton's algorithm will converge. But normally, if in the algorithm $x$ gets near the root, then the convergence is very fast. Actually it is the fastest method in the close neighborhood of the root. The number of exact digits nearly doubles after each iteration.

We can also see that if $f'(x)$ vanishes or is very small, then Newton's method fails. In this situation, it may be necessary to resort to bracketing methods.

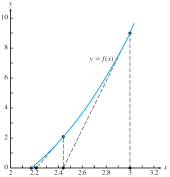
# Illustrating Newton's method

Now we illustrate Newton's method by locating a root of $x^3 + x = 2x^2 + 3$. We apply the method to the function $f(x) = x^3 - 2x^2 + x - 3$, starting with $x_0 = 3$. Of course, $f'(x) = 3x^2 - 4x + 1$, and these two functions should be arranged in nested form for efficiency:

$$\begin{aligned} f(x) &= ((x-2)x+1)x-3 \\ f'(x) &= (3x-4)x+1. \end{aligned}$$

Notice the doubling of the accuracy in $f(x)$ (and also in $x$) until the maximum precision of the computer is encountered:

| $k$ | $x_k$ | $f(x_k)$ |
|---|---|---|
| 0 | 3.0 | 9.0 |
| 1 | 2.4375 | 2.04 |
| 2 | 2.21303 27224 73144 5 | 0.256 |
| 3 | 2.17555 49386 14368 4 | $6.46 * 10^{-3}$ |
| 4 | 2.17456 01006 55071 4 | $4.48 * 10^{-6}$ |
| 5 | 2.17455 94102 93284 1 | $1.97 * 10^{-12}$ |



# Convergence analysis of Newton's method

## Theorem (convergence of Newton's method)

Suppose that $f$ is 2nd-order differentiable on $[r-\delta, r+\delta]$, $f(r)=0$ and $f'(x) \neq 0$ for all $x \in [r-\delta, r+\delta]$. Let

$$C = \frac{\sup\limits_{\xi \in [r-\delta, r+\delta]} |f''(\xi)|}{2 \inf\limits_{x \in [r-\delta, r+\delta]} |f'(x)|}.$$

Suppose further that

$$C\delta \leq 1.$$

If the initial point $x_0 \in (r-\delta, r+\delta)$, then all subsequent points $x_k$ obtained by Newton iteration are also in $(r-\delta, r+\delta)$, and converge to $r$ quadratically; that is,

$$|x_{k+1} - r| \leq C|x_k - r|^2.$$

## Proof.

Suppose $x_k \in (r-\delta, r+\delta)$. Then by Taylor's theorem, we have

$$0 = f(r) = f(x_k) + f'(x_k)(r - x_k) + \frac{f''(\xi_k)}{2}(r - x_k)^2,$$

and hence

$$\frac{f(x_k)}{f'(x_k)} + r - x_k + \frac{f''(\xi_k)}{2f'(x_k)}(r - x_k)^2 = 0,$$

$$r - x_{k+1} + \frac{f''(\xi_k)}{2f'(x_k)}(r - x_k)^2 = 0,$$

$$x_{k+1} - r = \frac{f''(\xi_k)}{2f'(x_k)}(x_k - r)^2,$$

$$|x_{k+1} - r| \le C|x_k - r|^2.$$

Finally, since $C\delta \le 1$, it is assured that $|x_{k+1} - r| < |x_k - r|$, which means $x_{k+1}$ remains in $(r-\delta, r+\delta)$. Hence the result. □

# Quadratic convergence and its consequence

An algorithm based on iterations is said to have quadratic convergence, if for each $k$, $e_{k+1} \leq Ce_k^2$, where $e_k$ is the absolute error of the $k$th iteration (or an upper-bound of it), $C$ is a constant. It is easy to arrive at

$$Ce_k \leq (Ce_0)^{2^k}.$$

Then if you want to have $e_k \leq \eta$, we need to have

$$k \geq \log_2[\log_{Ce_0}(C\eta)].$$

This means, if you have quadratic converge, you can be as accurate as machine precision allows in unbelievably few iterations!

# Linear convergence and its consequence

An algorithm based on iterations is said to have linear convergence, if for each $k$, $e_{k+1} \leq \rho e_k$, where $e_k$ is the absolute error of the $k$th iteration (or an upper-bound of it), $0 < \rho < 1$ is a constant.

We then have

$$e_k \leq \rho^k e_0.$$

Then if you want to have $e_k \leq \eta$, we need to have

$$k \geq \log_{1/\rho}(\frac{e_0}{\eta}).$$

The superiority of quadratic over linear convergence lies in the fact that $\log_{Ce_0}(C\eta)$ is much smaller than $\frac{e_0}{\eta}$.

# Superlinear convergence and its consequence

An algorithm based on iterations is said to have superlinear convergence, if for each $k$, $Ce_{k+1} \leq (Ce_k)^\alpha$, where $\alpha > 1$, $e_k$ is the absolute error of the $k$th iteration (or an upper-bound of it), $C$ is a constant. It is easy to arrive at

$$Ce_k \leq (Ce_0)^{\alpha^k}.$$

Then if you want to have $e_k \leq \eta$, we need to have

$$k \geq \log_\alpha[\log_{Ce_0}(C\eta)].$$

Usually we have $\alpha < 2$, for otherwise, we would call it quadratic or superquadratic. In numerical analysis, superquadratic convergence is very rare. Clearly, superlinear lies between linear and quadratic, and $\alpha$ is the bigger the better.

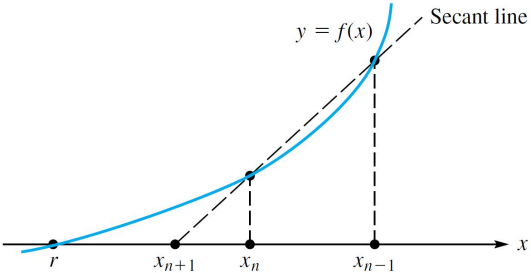
# Secant method

We now consider another procedure that converges almost as fast as Newton's method. This method mimics Newton's method but avoids the calculation of derivatives. In the secant method, we replace $f'(x_k)$ in Newton's iteration formula by an approximation that is easily computed. Specifically, we use

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Thus the secant iteration formula is

$$x_{k+1} = x_k - \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})} f(x_k).$$



# Secant method

**Algorithm 2.4:** Secant

**Input:** $f, x_0, x_1, M_{axiter}$

**Output:** $x$ which is an approximate zero of $f$

```
k = 0
f_0 = f(x_0)
while k < M_axiter do
    f_1 = f(x_1)
    x = x_1 - (x_1 - x_0)/(f_1 - f_0) * f_1
    x_0 = x_1
    x_1 = x
    f_0 = f_1
    k = k + 1
```

There is no guarantee that Secant algorithm will converge. The advantages of the secant method are that (after the first step) only one function evaluation is required per step (in contrast to Newton's iteration, which requires two) and that it is almost as rapidly convergent.

We can also see that if $f(x_k) - f(x_{k-1})$ vanishes or is very small, then Secant method fails. In this situation, it may be necessary to resort to bracketing methods.

# Illustrating secant method

## Example

If the secant method is used on $f(x) = x^5 + x^3 + 3$ with $x_0 = -1$ and $x_1 = 1$, what is $x_8$?

The output from the computer program corresponding to the pseudocode for the secant method (we use single precision) is as follows.

| $k$ | $x_k$ | $f(x_k)$ |
|---|---|---|
| 0 | -1.0 | 1.0 |
| 1 | 1.0 | 5.0 |
| 2 | -1.5 | -7.97 |
| 3 | -1.05575 | 0.512 |
| 4 | -1.11416 | $-9.991 * 10^{-2}$ |
| 5 | -1.10462 | $7.593 * 10^{-3}$ |
| 6 | -1.10529 | $1.011 * 10^{-4}$ |
| 7 | -1.10530 | $2.990 * 10^{-7}$ |
| 8 | -1.10530 | $2.990 * 10^{-7}$ |

# Convergence of secant method

Convergence analysis of the secant method is available in the literature. The rapidity of convergence of this method is, in general, between those for bisection and for Newton's method. The order of convergence of the secant method is $\frac{1+\sqrt{5}}{2} \approx 1.62$. Therefore this method has superlinear convergence.

Detailed analysis is omitted due to its complexity.

# Comparison between Regula Falsi and Secant

Both methods use secant. But they are different in many points.

- Regula Falsi uses solely interpolation, while in the secant method, extrapolation is much more common.
- Regula Falsi is for bracketing while the secant method is not.
- Regula Falsi can be used far away from the root, while the secant method has good behavior only near the root.

# Hybrid Methods

In an effort to find the best algorithm for finding a zero of a given function, various hybrid methods have been developed. Some of these procedures combine the bisection method (used during the early iterations) with either the secant method or the Newton method. Also, adaptive schemes are used for monitoring the iterations and for carrying out stopping rules.

Brent's Method is a hybrid method. It combines the property of guaranteed convergence, from the Bisection Method, with the property of fast convergence from the more sophisticated methods. It was originally proposed by Dekker and Van Wijngaarden in the 1960s, and improved by Brent later. MATLAB's command fzero implements a version of Brent's Method, along with a preprocessing step, to discover a good initial bracketing interval if one is not provided by the user.

Chandrupatla's method is both simpler than Brent's method, and converges faster for functions that are flat around their roots (which means they have multiple roots or closely-located roots). Basically it uses either bisection or inverse quadratic interpolation, based on a relatively simple criteria. Inverse quadratic interpolation is just quadratic interpolation using the y-values as inputs and the x-value as output.